

LAPORAN TUGAS PEMROSESAN TEKS & REPRESENTASI TEKS

Nama : Muhammad Razzan Carveya Hibrizi

NIM : 202210370311445

Kelas : NLP (C)

Source Code : <https://github.com/razzanch/Natural-Language-Processing.git>

STUDENT ASSIGNMENT: TEXT PROCESSING TASK

1. Choose a Dataset

Dataset yang digunakan adalah hasil scraping web dengan topik eksplorasi bahasa pemrograman dan framework yang diprediksi akan tetap relevan di masa depan pengembangan web. Dataset ini berisi dua kolom: URL dan Cleaned Text. Meskipun dataset sudah melalui proses pra-pemrosesan awal, langkah pra-pemrosesan tambahan tetap dilakukan untuk memastikan kualitas data.

Struktur Folder

datasets/

 cleaned_webdev_text_samples.csv

 processed_webdev_text.csv

preprocessing/

 text_preprocessing.py

vectorization/

 text_vectorization.py

main.py

Output Terlampir:

- BoW dan TF-IDF menghasilkan matriks yang sangat jarang (banyak nilai 0).
- Word Embeddings menghasilkan vektor representasi yang menangkap hubungan semantik antara kata-kata.

2. Preprocess the Text

Proses pra-pemrosesan teks mencakup langkah-langkah berikut:

- Convert to Lowercase: Mengubah semua teks menjadi huruf kecil.
- Remove Punctuation and Stopwords: Menghapus tanda baca, angka, URL, dan stopwords untuk mengurangi noise dalam teks.
- Tokenization and Lemmatization: Memecah teks menjadi token (kata-kata) dan mengubah kata-kata menjadi bentuk dasar (lemma).

Implementasi pra-pemrosesan dapat ditemukan dalam file `text_preprocessing.py`.

3. Convert Text to Numerical Form

Teks dikonversi ke bentuk numerik menggunakan tiga teknik berbeda:

- Bag of Words (BoW): Menghitung frekuensi kemunculan setiap kata dalam dokumen.
- TF-IDF (Term Frequency-Inverse Document Frequency): Memberikan bobot pada kata-kata berdasarkan pentingnya kata tersebut dalam dokumen dan korpus secara keseluruhan.
- Word Embeddings (Word2Vec): Menghasilkan representasi vektor untuk setiap kata berdasarkan konteksnya dalam teks.

Implementasi konversi teks ke bentuk numerik dapat ditemukan dalam file `text_vectorization.py`.

4. Analyze and Interpret

A. Discuss Which Technique Gives the Best Representation

Dari hasil output yang diberikan, teknik Word Embeddings (Word2Vec) memberikan representasi terbaik dibandingkan dengan Bag of Words (BoW) dan TF-IDF. Berikut adalah penjelasan rinci mengapa Word Embeddings lebih unggul:

1. Representasi Semantik

- Word Embeddings (Word2Vec): Teknik ini menangkap makna semantik dari kata-kata berdasarkan konteksnya dalam teks. Misalnya, kata “able” dan “use” memiliki vektor representasi yang mencerminkan hubungan mereka dalam konteks bahasa pemrograman dan pengembangan web. Kata-kata yang serupa secara makna (misalnya, “web” dan “internet”) akan memiliki vektor yang dekat satu sama lain dalam ruang vektor.

- BoW dan TF-IDF: Kedua teknik ini hanya mempertimbangkan frekuensi kemunculan kata tanpa memperhatikan makna atau hubungan antar kata. Mereka tidak dapat menangkap hubungan semantik seperti Word Embeddings.

Bag of Words (BoW) Sample:											
	ability	able	academy	accept	access	...	youll	youre	youtube	youve	zero
0	0	0	0	0	0	...	0	0	0	0	0
1	0	0	0	0	0	...	0	0	0	0	0
2	0	2	0	0	2	...	0	0	0	0	0
3	0	0	0	0	0	...	0	0	0	0	0
4	2	6	0	0	2	...	0	0	0	0	0
5	0	0	0	0	0	...	0	0	0	0	0
6	0	0	0	0	0	...	0	0	0	0	1
7	0	0	0	0	0	...	0	0	0	0	0
8	1	0	0	0	1	...	0	0	0	0	0
9	0	0	0	0	0	...	0	0	0	0	0

[10 rows x 2000 columns]

TF-IDF Sample:									
	ability	able	academy	accept	...	youre	youtube	youve	zero
0	0.000000	0.000000	0.0	0.0	...	0.0	0.0	0.0	0.000000
1	0.000000	0.000000	0.0	0.0	...	0.0	0.0	0.0	0.000000
2	0.000000	0.011611	0.0	0.0	...	0.0	0.0	0.0	0.000000
3	0.000000	0.000000	0.0	0.0	...	0.0	0.0	0.0	0.000000
4	0.011262	0.031473	0.0	0.0	...	0.0	0.0	0.0	0.000000
5	0.000000	0.000000	0.0	0.0	...	0.0	0.0	0.0	0.000000
6	0.000000	0.000000	0.0	0.0	...	0.0	0.0	0.0	0.096571
7	0.000000	0.000000	0.0	0.0	...	0.0	0.0	0.0	0.000000
8	0.021818	0.000000	0.0	0.0	...	0.0	0.0	0.0	0.000000
9	0.000000	0.000000	0.0	0.0	...	0.0	0.0	0.0	0.000000

2. Dimensi Tetap

- Word Embeddings (Word2Vec): Menghasilkan vektor dengan dimensi tetap (misalnya, 100 dimensi), yang jauh lebih kecil dibandingkan dengan jumlah fitur yang dihasilkan oleh BoW atau TF-IDF (dalam kasus ini, hingga 2000 fitur). Dimensi tetap membuat Word Embeddings lebih efisien untuk analisis lebih lanjut, seperti klasifikasi teks atau clustering.

```
Word2Vec Embedding Example:
[('data', array([ 0.05435001, -1.5020915,  0.87584496, -0.12677538, -0.6018451,
-2.1864192, -0.45768556, -0.37283954,  0.8396035,  0.6002387,
 0.9260408,  1.2205306, -1.8894973,  2.231897, -0.46381563,
-1.9872273, -3.1327076,  1.3718326,  0.7288302, -1.9879081,
 3.3594275,  1.6108311,  3.5819728,  1.1883345, -0.3385826,
-1.2819159, -1.0863589,  1.0247365, -1.4963766, -0.42467847,
-0.79569006, -0.82774746, -0.23368742, -1.8846834, -1.6012893,
 1.7650406,  0.1747908, -0.6546056,  0.13612176,  0.18732832,
-1.991787, -1.0570605, -2.4990988,  2.382156,  0.7112321,
 0.05901176, -0.30155873,  0.97197074,  1.5787925, -0.94491875,
-1.0409856, -0.8136941, -1.5006435,  1.5750544, -0.15087248,
 3.9678943,  2.1621592, -0.11169463,  1.4367151,  1.7205745,
-1.1040887, -0.53386605, -0.1613607, -1.2815262,  2.0273082,
-0.01915438, -2.7415712,  0.36261082, -1.8774366, -1.288395,
 1.7043438,  0.60094106,  0.2743906, -0.2940021,  3.0685961,
-0.5300335,  1.7321453,  1.6210248, -0.39638588, -0.2780417,
-0.20178142,  0.65203017, -2.6125894,  1.8857058, -1.6364875,
-0.04944403,  0.12939982,  0.46889222,  0.7041152,  1.1004697,
 3.047557, -3.7815669,  0.72359115, -0.24875604, -1.679286,
 3.087425,  0.62408626, -2.2694182,  0.39974663, -1.0562099 ])
```

- BoW dan TF-IDF: Menghasilkan matriks yang sangat besar (sparse matrix) karena setiap kata unik menjadi fitur tersendiri. Ini dapat menyebabkan masalah “curse of dimensionality,” terutama jika dataset memiliki banyak kata unik.

3. Menggunakan Konteks

- Word Embeddings (Word2Vec): Model Word2Vec dilatih berdasarkan konteks kata-kata, sehingga dapat menangkap hubungan antar kata. Misalnya, kata “web” dan “development” akan memiliki vektor yang serupa karena sering muncul bersama dalam teks tentang pengembangan web.
- BoW dan TF-IDF: Tidak memperhitungkan urutan atau konteks kata. Sebagai contoh, frase “pengembangan web” dan “web pengembangan” akan diwakili oleh vektor yang sama meskipun urutan kata berbeda.

B. Explain Why One Method Performs Better Than Another

Untuk memahami mengapa Word Embeddings lebih baik daripada BoW dan TF-IDF, kita perlu melihat kelemahan masing-masing teknik:

1. Kelemahan BoW

- Tidak Memperhitungkan Konteks atau Urutan Kata: BoW hanya menghitung frekuensi kemunculan kata tanpa memperhatikan urutan atau hubungan antar kata. Misalnya, frasa “pengembangan web” dan “web pengembangan” akan diwakili oleh vektor yang sama meskipun urutan kata berbeda.
- Matriks yang Sangat Jarang: BoW menghasilkan matriks yang sangat jarang (banyak nilai 0) karena sebagian besar kata hanya muncul sekali atau dua kali dalam dokumen. Ini membuatnya sulit digunakan untuk analisis lebih lanjut.

2. Kelemahan TF-IDF

- Tidak Memperhitungkan Hubungan Antar Kata: Meskipun TF-IDF memberikan bobot lebih tinggi pada kata-kata penting, metode ini tetap tidak memperhitungkan hubungan semantik antar kata. Sebagai contoh, kata “data” dan “information” mungkin dianggap sebagai entitas yang sepenuhnya berbeda meskipun memiliki makna serupa.
- Masih Menghasilkan Matriks yang Jarang: Seperti BoW, TF-IDF juga menghasilkan matriks yang jarang, yang dapat menyebabkan masalah dimensi tinggi.

3. Kelebihan Word Embeddings

- Menangkap Hubungan Semantik: Word Embeddings memahami hubungan antar kata berdasarkan konteks. Misalnya, kata “data” dan “information” akan memiliki vektor yang mirip karena sering muncul dalam konteks yang sama.
- Menghasilkan Vektor yang Lebih Informatif: Dibandingkan dengan BoW dan TF-IDF, Word Embeddings menghasilkan vektor yang lebih informatif karena mencerminkan makna semantik dan hubungan antar kata.
- Dimensi Tetap: Word Embeddings menghasilkan vektor dengan dimensi tetap, yang membuatnya lebih efisien untuk tugas-tugas machine learning seperti klasifikasi teks, clustering, atau analisis sentimen.

Kesimpulan

Dari analisis di atas, Word Embeddings (Word2Vec) adalah teknik terbaik untuk merepresentasikan teks dalam dataset ini karena:

- Menangkap Makna Semantik: Word Embeddings menangkap hubungan semantik antar kata berdasarkan konteks.
- Dimensi Tetap: Menghasilkan vektor dengan dimensi tetap yang lebih kecil dibandingkan dengan BoW atau TF-IDF.
- Lebih Cocok untuk Tugas Lanjutan: Word Embeddings lebih cocok untuk tugas-tugas seperti klasifikasi teks, clustering, atau analisis sentimen karena representasinya yang informatif.

Namun, BoW dan TF-IDF masih berguna dalam beberapa kasus, terutama jika:

- Dataset relatif kecil.
- Tujuan utamanya adalah analisis sederhana seperti menghitung frekuensi kata.

Jawaban Akhir:

Word Embeddings (Word2Vec) memberikan representasi terbaik karena menangkap makna semantik, hubungan antar kata, dan menghasilkan dimensi fitur yang lebih kecil.